Put your POS online. One step at a time.

A simple guide for **Namecheap Stellar Plus**. One subdomain now; add another later.

Shop browser → React website

↓

NestJS API → MySQL database

↓

Shop print agent → Receipt printer

Can Stellar Plus host this system?

It supports the basic hosting tools you need. Use the steps below to prove your application works on your assigned server before using it for real sales. Hosting support alone does not guarantee POS performance.

One subdomain for Tifoam

Website: `https://tifoam.prosincsoftware.com`

API: `https://tifoam.prosincsoftware.com/api`

Main domain: prosincsoftware.com. Create only tifoam.prosincsoftware.com now. The /api part is a path on that subdomain, not another subdomain.

Prepared 6 October 2026 • Project files reviewed; hosting account and live database were not accessed. No passwords or shop data are included.

UNDERSTAND YOUR ENVIRONMENT

What goes where?

Your folder	What it does	Where it runs
frontend React 19 + Vite 7	Screens staff use in the browser.	Copy frontend/dist contents into the backend public/ folder; one Node app serves both.
backend NestJS 11 + TypeORM	Login, products, sales, refunds, purchasing, stock and reports.	Private server folder, managed by cPanel's Node.js app feature.
MySQL database	Users, settings, products, sales and stock records.	A database created in cPanel; it is not a website folder.
print-agent	Collects print jobs and sends them to the receipt printer.	The shop's Windows computer, beside the printer.

Your current development setup

The project normally runs its API at `http://localhost:3000/api` and its development website at `http://localhost:5173`. Online, these become HTTPS domain addresses. Staff must not keep using localhost for the online API.

What you need

- A domain, Stellar Plus account and access to cPanel.
- A current project copy, Node.js 22.12+ locally, and internet at the shop. Have a developer complete the single-app setup on page 10 before building.
- A full database export if keeping existing records.
- A copy of `backend/uploads` if present, for uploaded logos.
- Your admin login and a printer/Windows driver if printing receipts.

> **Keep these private:** backend .env, SQL backups, passwords and print-agent tokens. Never upload them into a public website folder. Do not copy Windows node_modules to the Linux server.

Prepare your hosting account

1. Confirm the server can run your POS

Before buying, or before uploading, send Namecheap Support this message:

> I want to host a NestJS 11 API with MySQL and a static React website on Stellar Plus. Please confirm Node.js 22 or 24 is available in Setup Node.js App, SSH/Terminal can be enabled, and tell me the database server version, Node app memory limits and app log location. My SQL uses utf8mb4_0900_ai_ci. Is this supported on my assigned server?

Your SQL baseline uses a MySQL 8 collation. If the assigned server does not support it, have a developer prepare and test a compatible import and migrations first. Do not assume MariaDB is identical to MySQL 8.

2. Connect your domain and open cPanel

1. Save Namecheap's hosting welcome email.
2. In Namecheap, open **Domain List → Manage → Nameservers**. For a domain managed there, select **Namecheap Web Hosting DNS**. For another registrar, follow the welcome email's DNS instructions.
3. Keep the main domain and its existing email records working. Hosting one subdomain does not require moving the entire domain.
4. Open **Hosting List → Go to cPanel**. Wait for the domain to resolve to the hosting server.

3. Create ONE subdomain and enable HTTPS

In **cPanel ? Domains**, create `tifoam.prosincsoftware.com`. Give it its own document root, separate from your main domain. Do not select a shared document root.

Point this subdomain to your hosting server using the DNS provider that manages your domain. If your main domain is already hosted elsewhere, keep it there and ask Support to help point only the tifoam subdomain.

Activate SSL for this subdomain. Both the website and /api use this one certificate. Confirm HTTPS works before entering passwords.

> **Hosting check:** Namecheap documents Node.js apps on shared hosting, including Node 22 and 24. The options on your assigned server are the final check. [1–3]

STEPS 4–5 / DATABASE

Move your data safely

4. Create the online database

1. Open **cPanel** → **Manage My Databases** or **MySQL Database Wizard**.
2. Create a database, for example `bakery_pos`.
3. Create a database user with a strong password.
4. Add that user to that database and grant **All Privileges** on that database.
5. Record the full names shown, including the cPanel prefix, such as `account_bakery_pos` and `account_posuser`. Confirm the database host with Support. [4]

5. Choose ONE database path

A. Keep your existing products, users and sales

Export your current database as SQL with both structure and data. Include its `migrations` table. Keep an untouched backup.

In cPanel → phpMyAdmin, select the new database → Import → select the SQL file → Go. The export should target the selected database, without creating or switching to your old database name. If too large, ask Support for an SSH import. [5–6]

After installing the API files, run the migration commands on page 6. **Do not import the baseline on top of this data.**

B. Start with no existing shop records

Only into the empty database, import `backend/schema/baseline-1770000023000.sql` using phpMyAdmin. This file creates the initial tables and migration history, but contains no users or products.

Then run later migrations on page 6 and initialize the first administrator on page 7.

Important for your project: Do not run `npm run db:bootstrap` blindly on shared hosting. Its script issues CREATE DATABASE, which your cPanel database user may not be allowed to run. Create the database in cPanel, then use the import path above. Keep `DB_SYNCHRONIZE=false`.

STEPS 6–7 / API FILES

Build and upload the backend

6. Build on your own computer

Open PowerShell in your project folder. Run these commands one at a time:

```
cd backend
npm ci
npm run build
```

Continue only if the build finishes successfully and `backend/dist/main.js` exists. This compiles TypeScript into the JavaScript used by the server.

First have a developer complete page 10. Prepare an archive with `dist/`, `package.json`, `package-lock.json` and existing `uploads/`. Add the website in `public/` as explained on page 7. Keep the original source and database backup on your computer. The runtime upload does not need node_modules or your local .env.

7. Upload and create the Node.js application

In File Manager create a folder outside public website roots, such as `/home/ACCOUNT/tifoam-app`. Replace ACCOUNT with your cPanel username. Upload and extract the archive there. Files must be directly inside tifoam-app, not hidden inside an extra backend folder.

Create `app.js` in that folder with this content:

```
require('./dist/main.js');
```

In **cPanel → Setup Node.js App → Create Application**, use:

Field	Example value
Node.js version	22 (current available patch) or 24
Application mode	Production
Application root	tifoam-app
Application URL	tifoam.prosincsoftware.com at root /
Startup file	app.js

Use the cPanel app manager to run/restart the app. This hosting recipe needs a staging check with your assigned Node.js environment. Do not start a separate background Node process. [1]

STEPS 8–9 / API CONFIGURATION

Connect the API to MySQL

8. Add production settings

Add these environment variables in the Node.js app settings. Use your actual full database names and password:

```
NODE_ENV=production
DB_HOST=localhost
DB_PORT=3306
DB_USERNAME=account_posuser
DB_PASSWORD=YOUR_STRONG_DATABASE_PASSWORD
DB_DATABASE=account_bakery_pos
DB_SYNCHRONIZE=false
```

Use the host supplied by Support if it is different. Do not copy your local PORT setting into cPanel; the host manages application routing. Your API already reads the host's PORT value when provided.

Use the app manager's **Run NPM Install**. For migrations, enable SSH/Terminal if needed, copy the activation command shown in the Node.js app screen, and run it exactly as shown. Confirm you are in `tifoam-app`.

> **Terminal settings:** The TypeORM CLI reads process environment variables or .env. Do not assume the app manager's variables are present in Terminal. Before migrating, ensure all DB_* variables above are available there. A private .env inside tifoam-app is an alternative; keep it outside public roots, readable only by your hosting account, and consistent with the app settings.

9. Apply pending migrations and restart

Back up an existing database first. Because you uploaded compiled files, use these commands rather than scripts that rebuild missing source files:

```
node node_modules/typeorm/cli.js migration:show -d dist/data-source.js
node node_modules/typeorm/cli.js migration:run -d dist/data-source.js
node node_modules/typeorm/cli.js migration:show -d dist/data-source.js
```

Stop if a command fails. Investigate the error before retrying. When no migrations remain pending, restart the app in cPanel.

Open `https://tifoam.prosincsoftware.com/api/auth/login` in a browser. A GET request may return 404 because login expects POST; this alone is not a failure. Verify login through the website in the next step and inspect the app log for startup/database errors.

STEPS 10–11 / WEBSITE

Upload the screens staff use

10. Build the frontend for your online API

Open a new PowerShell window in the project folder:

```
cd frontend
$env:VITE_API_URL = 'https://tifoam.prosincsoftware.com/api'
npm ci
npm run build
```

This API address is saved into the built website. When changing it later, rebuild and upload again. It is not a secret.

On your computer create `backend/public`. Copy the **contents** of `frontend/dist` into it: `index.html`, `assets/` and the other generated files.

Upload this `public/` folder into the private `tifoam-app` folder alongside `dist/` and `app.js`. Restart the Node.js app.

```
/home/ACCOUNT/tifoam-app/
  app.js
  package.json + package-lock.json
  dist/          compiled API
  public/        index.html + assets/
  uploads/       existing uploaded logos
```

The developer setup on page 10 serves the website at `/` and the API at `/api`. It also handles refreshing React pages. Leave cPanel's generated routing configuration in place.

11. Sign in and configure the business

If you imported existing data: use your existing admin account. Check users, products, balances and a historical sale.

If you chose a fresh database: have your developer initialize the first platform admin before public access. The project has `POST /api/auth/bootstrap` accepting username and a password of at least 8 characters. A stronger password is recommended. Both bootstrap routes reject creation after a platform user exists; keep the empty installation private until initialization is complete.

Then configure the tenant, locations, staff/roles, products, price lists, payments and registers. Confirm the tenant timezone is `Asia/Colombo` for this Sri Lankan shop.

> **Developer check before launch:** Website and API now share the same origin. Your existing API enables unrestricted CORS; a developer can remove it for this same-origin setup. Verify login, permissions and tenant isolation before launch.

STEP 12 / SHOP PRINTER

Keep printing at the shop

> **Correction needed before automatic printing:** Your backend uses the global `/api` prefix, but `print-agent/agent.mjs` requests `/pos-print/agent/...`. Ask your developer to change claim and complete URLs to `/api/pos-print/agent/...` and test them. Setting serverUrl to end in /api will not fix this: the leading slash in the request replaces that path. This guide does not modify the agent.

12. Set up the local print agent

1. On the shop computer install Node.js 22 or newer and the printer driver.
2. In the POS open **Register Management** → **Terminals** → **Receipt printer**.
3. Select **WINDOWS_QUEUE** for a USB/Windows printer, or **TCP** for a network ESC/POS printer. Enter the exact Windows queue name or local printer IP/port, and paper width.
4. Save the profile. Copy its profile ID and one-time token.
5. Copy the corrected print-agent folder to the shop computer. Create `agent-config.json` inside it:

```
{
  "serverUrl": "https://tifoam.prosincsoftware.com/",
  "profileId": 12,
  "agentToken": "REPLACE_WITH_YOUR_64_CHARACTER_TOKEN",
  "statePath": "C:/BakeryPOS/print-state.json"
}
```

Replace 12 with your profile ID. Create `C:/BakeryPOS` first. Keep this configuration private. In the print-agent folder run:

```
npm ci
node agent.mjs agent-config.json
```

Use **Test print** and check Recent print jobs. For automatic startup, create a Windows Task Scheduler task at logon: program = full path to node.exe; arguments = `agent.mjs agent-config.json`; Start in = full print-agent folder. Enable restart on failure.

Simple alternative: use the receipt screen's **Browser print** button and select the printer in the Windows print dialog. This does not require the local agent.

The shop needs internet to reach the hosted API. Do not expose your printer port to the internet. Cloud hosting does not make this system an offline POS.

STEPS 13–14 / LAUNCH & CARE

Test first. Then go live.

13. Complete this launch checklist

- ☐ The tifoam website and its /api routes have valid HTTPS.
- ☐ Admin and cashier logins work; permissions behave correctly.
- ☐ A page still works after refreshing its URL.
- ☐ Products, prices, stock, users and logos are correct.
- ☐ On a separate test database, complete a sale, payment, receipt, refund and stock check; verify totals and daily bill numbering.
- ☐ Test real printer output and restarting the shop computer.
- ☐ Check response times with the expected number of tills and reports.
- ☐ Export a backup and successfully restore it to a separate test database.

For the final move, stop transactions on the old system, export the latest data, import to the online database and apply migrations. Reconcile sales and stock before resuming. Use one live database so records do not split between two systems.

14. Back up and update carefully

Keep dated database exports and copies of `uploads/` outside the hosting account. Check Stellar Plus AutoBackup in cPanel; also keep your own backups. [2] Before updates, back up, test the release, upload dist/ and public/, run pending migrations and restart. If an update fails, stop new transactions and restore code/data together with developer help; do not overwrite new sales with an old backup.

Problem	First check
503 / API will not start	App logs, app.js, dependencies, DB credentials and account resource usage.
Website opens but login fails	Built VITE_API_URL, /api routing, HTTPS and API logs.
Unknown collation / import fails	Database version and tested SQL compatibility.
Printing stays pending	Agent /api URL correction, running task, profile token and printer connection.

Official references, checked 6 October 2026

[1] Namecheap: Node.js application setup
[2] Namecheap: Shared hosting, DNS, SSL and backups
[3] Namecheap: Shared hosting plan comparison
[4] Namecheap: Create databases and users
[5] Namecheap: phpMyAdmin · [6] Namecheap: SQL import via SSH

DEVELOPER SETUP / ONE SUBDOMAIN

Serve the website and API together

Required before step 6. The current backend serves the API only. A developer must add the website handling below and test it. These are proposed instructions; this PDF does not change your application.

In `backend`, run `npm install express@^5`. Keep the updated package files. The project already includes Express types. Replace `backend/src/main.ts` with:

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { static as serveStatic } from 'express';
import type { Request, Response, NextFunction } from 'express';
import { join } from 'path';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.setGlobalPrefix('api');
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true, transform: true,
  }));
  const website = join(__dirname, '..', 'public');
  const files = serveStatic(website, { index: false });
  app.use((req: Request, res: Response, next: NextFunction) => {
    if (req.path === '/api' || req.path.startsWith('/api/')) {
      return next();
    }
    files(req, res, (error?: unknown) => {
      if (error) return next(error);
      if (req.method !== 'GET' || !req.accepts('html')) {
        return next();
      }
      res.sendFile(join(website, 'index.html'), (err) => {
        if (err) next(err);
      });
    });
  });
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Verify: build with `npm run build`, use the built frontend in public/, and test the website, asset files, a refreshed React page, API login, an unknown /api route and printing. API requests must return API responses, not the website HTML. Test the same routing on cPanel before real sales.

Another subdomain later

For another independent POS, create its subdomain, a separate Node app folder and database. Build that website using its own /api address. Keep Tifoam's folders and database separate.

Implementation reference: Express 5 static files and NestJS with Express.